



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERÍA

Escuela Profesional de Ingeniería de Sistemas

Proyecto Syntax Validator

Curso: Base de Datos II

Docente: Patrick Jose Cuadros Quiroga

Integrantes:

Arocutipa Arocutipa, Gian Franco (2023076790)

Soto Oquendo Cristian Gabriel (2026086510)

Tacna – Perú
2026

CONTROL DE VERSIONES					
Versión	Hecha por	Revisada por	Aprobada por	Fecha	Motivo
1.0	G.A. / C.S.			27/04/2026	Versión Original

Syntax Validator

Documento de Especificación de Requerimientos de Software

Versión {1.0}

CONTROL DE VERSIONES					
Versión	Hecha por	Revisada por	Aprobada por	Fecha	Motivo
1.0	G.A. / F.P.			27/04/2026	Versión Original

ÍNDICE GENERAL

INTRODUCCIÓN	5
I. Generalidades de la empresa	6
A. Nombre de la empresa	6
B. Visión	6
C. Misión	6
D. Organigrama	6
II. Visionamiento de la empresa	7
A. Descripción de la empresa	7
B. Objetivos de negocios	7
C. Objetivos de diseño	7
D. Alcance del proyecto	8
E. Viabilidad del sistema	8
F. Información obtenida del Levantamiento de Información	9
III. Análisis de Procesos	9
A. Diagrama del Proceso Actual - Diagrama de actividades	9
B. Diagrama del Proceso Propuesto - Diagrama de actividades Inicial	10
IV. Especificación de Requerimientos de Software	12
A. Cuadro de Requerimientos funcionales Inicial	12
B. Cuadro de Requerimientos No funcionales	12
C. Cuadro de Requerimientos funcionales Final	13
D. Reglas de Negocio	14
V. Fase de Desarrollo	14
A. Perfiles de Usuario	14
B. Modelo Conceptual	15
a. Diagrama de Paquetes	15
b. Diagrama de Casos de Uso	16
c. Escenarios Casos de Uso	17
C. Modelo Lógico	18
a. Análisis de Objetos	18
b. Diagrama de Actividades con objetos	19
c. Diagrama de Secuencia	36
d. Diagrama de Clases	42
CONCLUSIONES	43

RECOMENDACIONES	43
BIBLIOGRAFÍA	43

INTRODUCCIÓN

El presente documento describe la Especificación de Requerimientos de Software (SRS) para el sistema SQL/NoSQL Syntax Validator, una aplicación web orientada a la validación de sintaxis de consultas SQL y MongoDB con detección de errores en tiempo real, sugerencias de corrección automática y soporte para múltiples dialectos.

El sistema propone una interfaz inspirada en entornos de desarrollo (VS Code-like) con editor Monaco integrado, que permite a desarrolladores y estudiantes validar consultas de base de datos sin necesidad de conectarse a un motor real. El validador soporta los dialectos MySQL, PostgreSQL, SQLite y ANSI para SQL, así como comandos CRUD, de agregación, administración, sharding y replicación para MongoDB.

El sistema se desarrolla con arquitectura MVC, utilizando Node.js con Express en el backend y JavaScript puro (ES6+) en el frontend, con Monaco Editor como componente central de la interfaz de usuario.

I. Generalidades de la empresa

A. Nombre de la empresa

SQL/NoSQL Syntax Validator — Herramienta Web de Validacion de Consultas de Base de Datos

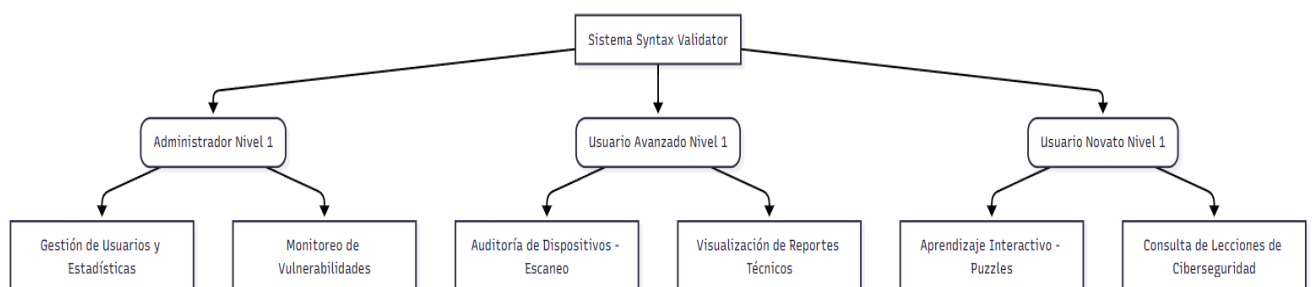
B. Visión

Ser la herramienta de referencia en validacion de sintaxis de consultas de base de datos para desarrolladores, estudiantes y arquitectos de datos en el ambito hispanoparlante, proporcionando retroalimentacion inmediata y precisa sin necesidad de un motor de base de datos real.

C. Misión

Proveer a la comunidad de desarrolladores y estudiantes una herramienta web ligera, precisa e intuitiva para validar, depurar y aprender la sintaxis correcta de SQL y MongoDB, reduciendo el tiempo de deteccion de errores sintaticos en el ciclo de desarrollo de software.

D. Organigrama



II. Visionamiento de la empresa

A. Descripción de la empresa

Los desarrolladores y estudiantes de base de datos frecuentemente cometen errores de sintaxis al escribir consultas SQL o MongoDB que solo son descubiertos al ejecutar el código contra un motor real. Este proceso requiere configurar entornos de base de datos, crear esquemas de prueba y gestionar conexiones, lo que incrementa el tiempo de desarrollo y dificulta el aprendizaje.

El SQL/NoSQL Syntax Validator elimina esta fricción al ofrecer un validador en tiempo real directamente en el navegador, con indicación precisa de la línea y columna del error, sugerencias de corrección y soporte para múltiples dialectos en una única herramienta.

B. Objetivos de negocios

Nº	Objetivo	Indicador de Exito
1	Validar consultas SQL con soporte multi-dialecto	Soporte de MySQL, PostgreSQL, SQLite y ANSI con detección correcta de al menos el 95% de errores sintáticos
2	Validar consultas MongoDB con detección semántica	Soporte de 20+ comandos y 10+ operadores con mensajes de error descriptivos
3	Proporcionar retroalimentación precisa de errores	Errores reportados con número de línea y columna exactos
4	Mantener historial de consultas del usuario	Historial persistente vía localStorage con acceso rápido
5	Soportar carga de archivos .sql	Carga correcta de archivos .sql de hasta 1MB en el editor

C. Objetivos de diseño

Nº	Objetivo de Diseno
1	Arquitectura MVC con separacion clara: Modelo (validation.service.js), Vista (index.html + app.js + styles.css), Controlador (validation.controller.js)
2	Backend Node.js/Express con endpoints REST: POST /api/validate, GET /api/examples, GET /api/health
3	Frontend con Monaco Editor para resaltado de sintaxis, autocompletado y marcado de errores en linea
4	Soporte de temas claro y oscuro con persistencia de preferencia del usuario
5	Diseno responsivo compatible con resoluciones desde 768px

D. Alcance del proyecto

Modulo	Descripcion	Tecnologia
API REST de Validacion	Recibe consultas, las valida y retorna errores con posicion exacta	Node.js + Express
Validador SQL	Soporte para MySQL, PostgreSQL, SQLite y ANSI con node-sql-parser	node-sql-parser v4
Validador MongoDB	Validacion de estructura JSON, comandos y operadores MongoDB	JSON.parse nativo + logica propia
Editor de Consultas	Editor Monaco integrado con resaltado, autocompletado y marcado de errores	Monaco Editor
Historial de Consultas	Persistencia local de validaciones anteriores para acceso rapido	localStorage
Carga de Archivos SQL	Importacion de archivos .sql desde el sistema de archivos local	File API del navegador
Tema Oscuro/Claro	Alternancia entre temas con persistencia de preferencia	CSS custom properties

E. Viabilidad del sistema

El sistema es viable tecnica, economica y operativamente:

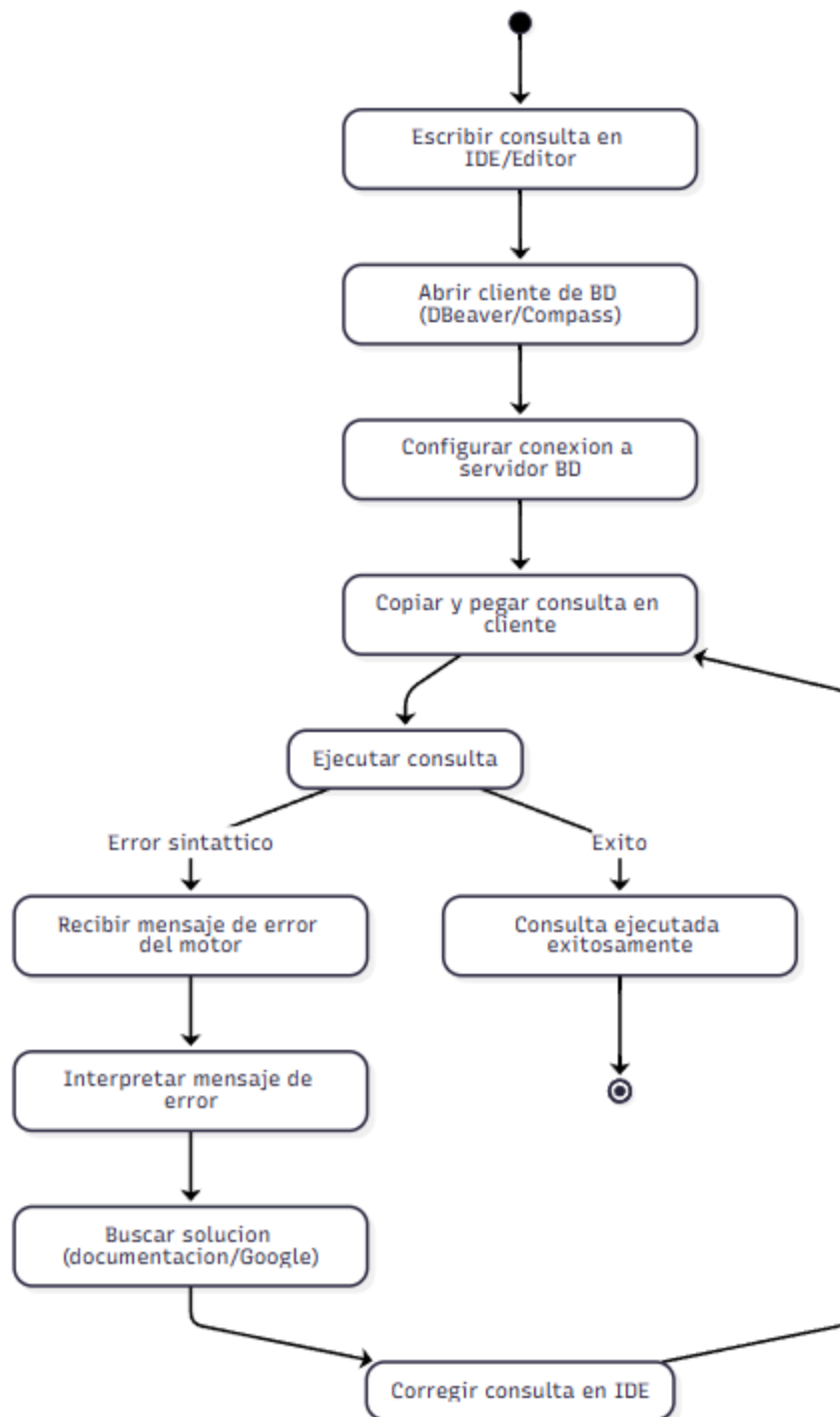
- **Tecnicamente:** El equipo domina Node.js, Express, JavaScript ES6+ y el patron MVC. Las dependencias principales (node-sql-parser, Monaco Editor) son de codigo abierto y activamente mantenidas.
- **Economicamente:** No se requieren licencias de software. El despliegue puede realizarse en plataformas gratuitas como Vercel, Railway o Render para el backend, y Netlify para el frontend.
- **Operativamente:** Al ser una aplicacion web, no requiere instalacion en el cliente. El backend es stateless, lo que facilita el escalado horizontal.

F. Información obtenida del Levantamiento de Información

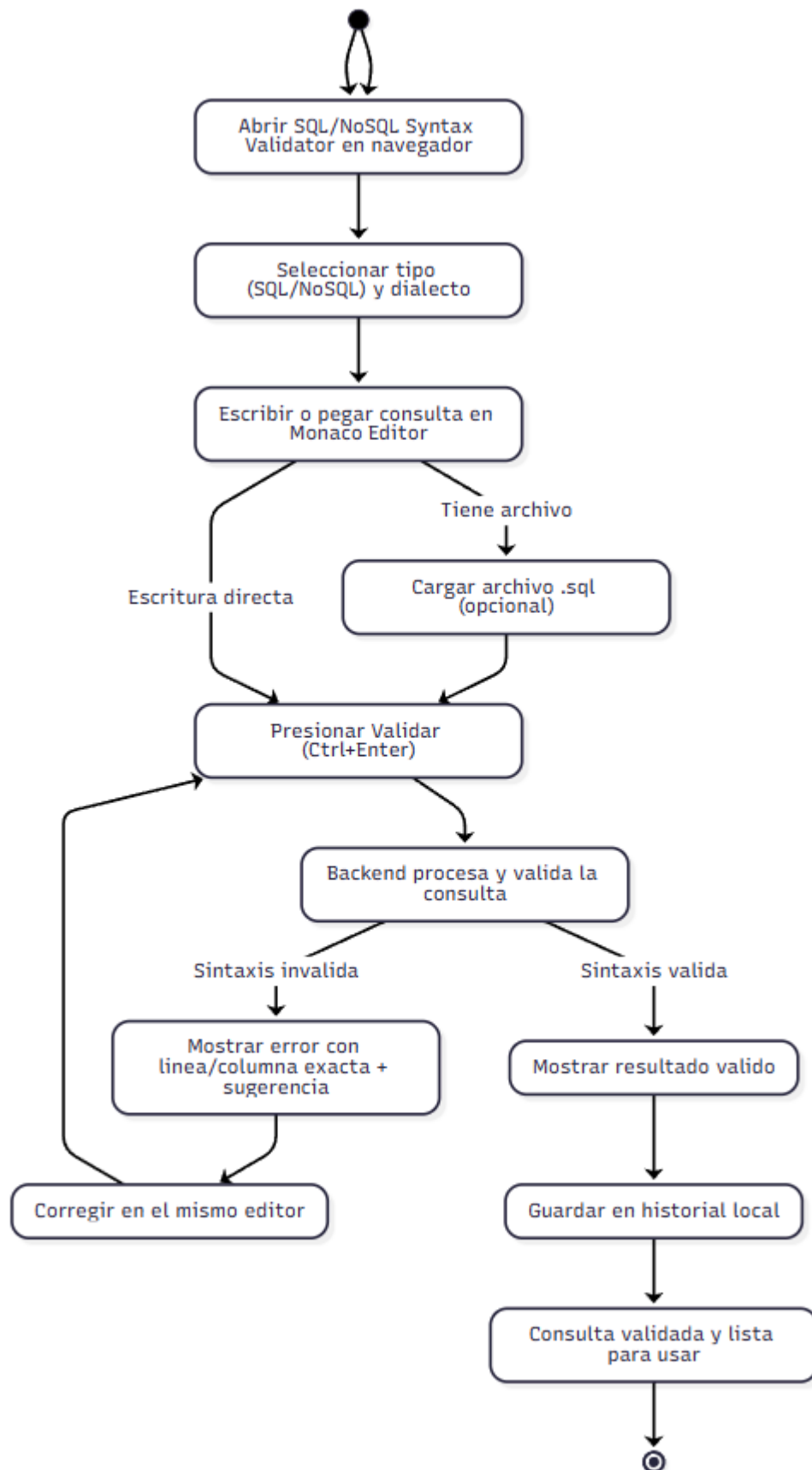
Tecnica	Fuente	Hallazgo Principal
Analisis de herramientas existentes	SQLFiddle, DB Fiddle, MongoDB Playground	Requieren conexion a internet con BD real; no detectan errores sin ejecutar
Revision de parsers SQL	npm registry, GitHub	node-sql-parser es el parser mas completo y mantenido para Node.js con soporte multi-dialecto
Encuesta informal	Estudiantes de IS/CC	70% prefiere validar sintaxis antes de ejecutar contra la BD de produccion
Analisis de Monaco Editor	Documentacion Microsoft	Monaco permite marcado de errores en linea con la API setModelMarkers, ideal para UX de validacion

III. Análisis de Procesos

A. Diagrama del Proceso Actual - Diagrama de actividades



B. Diagrama del Proceso Propuesto - Diagrama de actividades Inicial



Logo de Mi Empresa

Logo de mi Cliente

IV. Especificación de Requerimientos de Software

A. Cuadro de Requerimientos funcionales Inicial

ID	Modulo	Requerimiento Funcional	Prioridad
RF-01	Validacion SQL	El sistema debe validar sintaxis SQL con soporte para MySQL, PostgreSQL, SQLite y ANSI	Alta
RF-02	Validacion SQL	El sistema debe retornar la linea y columna exacta de cada error sintattico SQL	Alta
RF-03	Validacion SQL	El sistema debe proveer sugerencias de correccion para errores SQL comunes	Media
RF-04	Validacion NoSQL	El sistema debe validar estructura JSON valida en consultas MongoDB	Alta
RF-05	Validacion NoSQL	El sistema debe validar comandos MongoDB soportados (find, insert, update, delete, aggregate, etc.)	Alta
RF-06	Validacion NoSQL	El sistema debe validar operadores MongoDB (\$in, \$and, \$or, \$mod, \$text, etc.)	Alta
RF-07	Editor	El sistema debe ofrecer un editor Monaco con resaltado de sintaxis para SQL y JSON	Alta
RF-08	Historial	El sistema debe guardar cada validacion en el historial local del usuario	Media
RF-09	Archivos	El sistema debe permitir cargar archivos .sql para su validacion	Media
RF-10	API	El sistema debe exponer endpoint POST /api/validate para validacion programatica	Alta
RF-11	API	El sistema debe exponer endpoint GET /api/examples con consultas de ejemplo	Baja
RF-12	API	El sistema debe exponer endpoint GET /api/health con estado del servidor	Baja

B. Cuadro de Requerimientos No funcionales

ID	Categoria	Requerimiento No Funcional
RNF-01	Rendimiento	La validacion debe completarse en menos de 2 segundos para consultas de hasta 10,000 caracteres
RNF-02	Disponibilidad	El backend debe tener disponibilidad del 99% en entorno de produccion
RNF-03	Usabilidad	La interfaz debe ser intuitiva y no requerir manual de usuario para operaciones basicas
RNF-04	Compatibilidad	El frontend debe funcionar en Chrome 90+, Firefox 88+, Safari 14+ y Edge 90+
RNF-05	Mantenibilidad	El codigo debe seguir el patron MVC con separacion clara de capas y cobertura de pruebas unitarias $\geq 80\%$
RNF-06	Seguridad	El backend debe sanitizar todas las entradas para prevenir inyeccion de codigo malicioso
RNF-07	Portabilidad	El backend debe ejecutarse en Node.js v18+ en Linux, macOS y Windows
RNF-08	Escalabilidad	La arquitectura debe soportar al menos 100 peticiones/segundo sin degradacion de rendimiento
RNF-09	Configurabilidad	El puerto del servidor debe ser configurable via variable de entorno PORT

C. Cuadro de Requerimientos funcionales Final

ID	Caso de Uso	Actor	Descripcion Refinada	Prior.
RF-01	UC-01	Usuario	Seleccionar tipo SQL, elegir dialecto (MySQL/PostgreSQL/SQLite/ANSI), presionar Validar. Sistema invoca POST /api/validate con {type:'sql', query, dialect}.	Alta
RF-02	UC-01	Sistema	Parser node-sql-parser procesa la consulta y retorna {valid, errors:[{line,column,message}], suggestions}.	Alta
RF-03	UC-02	Usuario	Seleccionar tipo NoSQL, ingresar JSON MongoDB, presionar Validar. Sistema invoca POST /api/validate con {type:'nosql', query}.	Alta
RF-04	UC-02	Sistema	Sistema valida estructura JSON, comando MongoDB y operadores. Retorna errores especificos por tipo.	Alta
RF-05	UC-03	Usuario	Presionar 'Subir Archivo', seleccionar .sql. Editor carga contenido y cambia a modo SQL automaticamente.	Media
RF-06	UC-04	Usuario	Cada validacion se guarda en localStorage. Boton 'Historial' muestra lista de consultas anteriores.	Media
RF-07	UC-01/02	Sistema	Monaco Editor marca errores en la posicion exacta (linea/columna) mediante setModelMarkers.	Alta
RF-08	UC-05	Usuario	GET /api/examples retorna consultas SQL y MongoDB de ejemplo clasificadas por tipo.	Baja
RF-09	UC-06	Usuario/Dev	GET /api/health retorna estado, timestamp, version y uptime del servidor.	Baja

D. Reglas de Negocio

ID	Regla de Negocio	Descripcion
RN-01	Dialecto Obligatorio en SQL	Toda validacion SQL debe especificar un dialecto. El sistema usa MySQL como dialecto por defecto si no se especifica.
RN-02	JSON Valido Previo	Toda consulta MongoDB debe ser JSON valido antes de validar comandos y operadores. Si el JSON es invalido, no se realizan validaciones semanticas.
RN-03	Operadores Array	Los operadores \$in, \$nin, \$all, \$and, \$or, \$nor deben tener como valor un array. Si no es array, se reporta error semantico.
RN-04	Operador \$mod	El operador \$mod debe tener exactamente 2 elementos en su array: [divisor, residuo].
RN-05	Operador \$text	El operador \$text debe ser un objeto que contenga la propiedad \$search de tipo string.
RN-06	Historial Local	El historial se almacena en localStorage del navegador. No se sincroniza entre dispositivos ni se envia al servidor.
RN-07	Limite de Archivo	Solo se aceptan archivos con extension .sql para carga desde el sistema de archivos.

V. Fase de Desarrollo

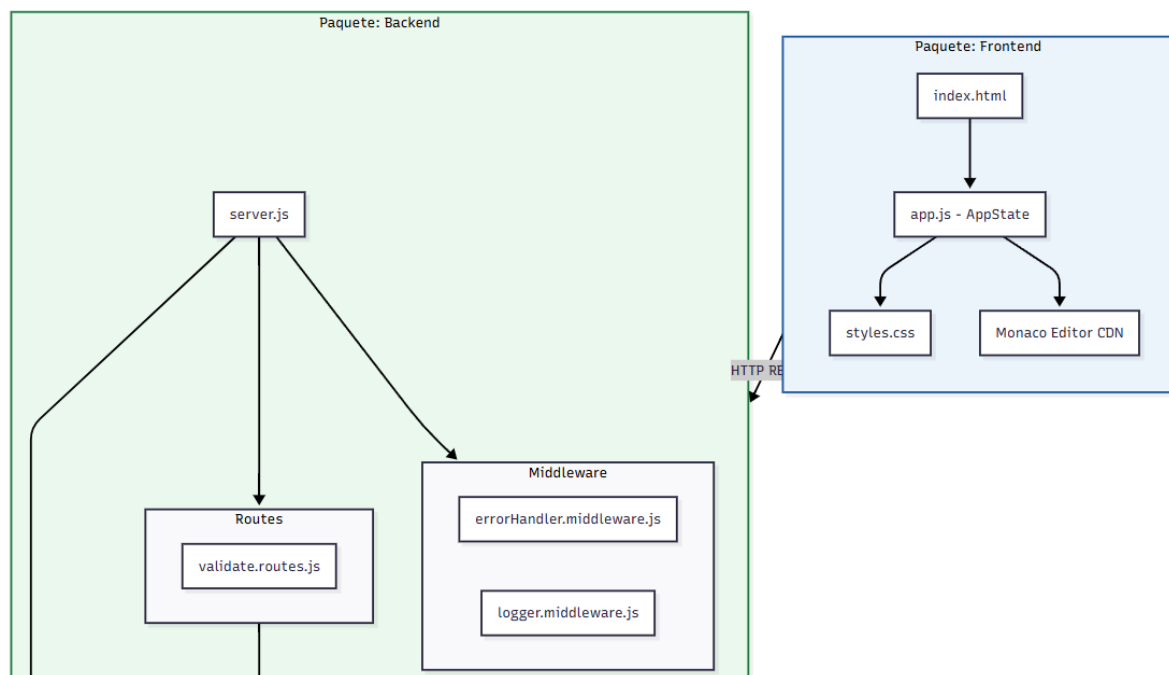
A. Perfiles de Usuario

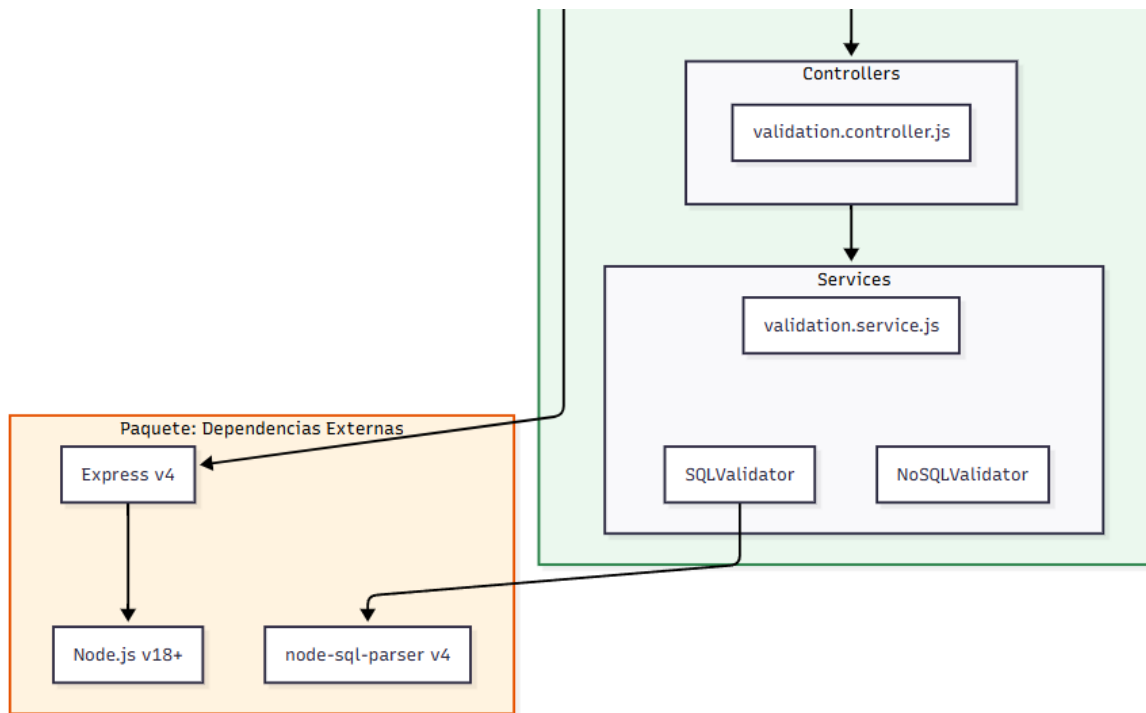
Perfil	Descripción	Acceso al Sistema
Desarrollador Backend	Profesional que escribe consultas SQL/MongoDB en su flujo de trabajo diario. Usa el validador para verificar sintaxis antes de ejecutar en produccion.	Editor, validacion SQL/NoSQL, historial, carga de archivos, API REST
Estudiante de BD	Estudiante de bases de datos que aprende SQL o MongoDB. Usa el validador para comprender errores y mejorar sus consultas con las sugerencias.	Editor, validacion SQL/NoSQL, historial, ejemplos precargados

Arquitecto de Datos	Profesional que disena esquemas y consultas complejas. Valida consultas multi-dialecto para garantizar portabilidad.	Editor multi-dialecto, API REST para integracion CI/CD
Sistema (API Consumer)	Aplicacion externa que consume la API REST del validador para integrar validacion en su propio flujo de CI/CD.	POST /api/validate, GET /api/health, GET /api/examples

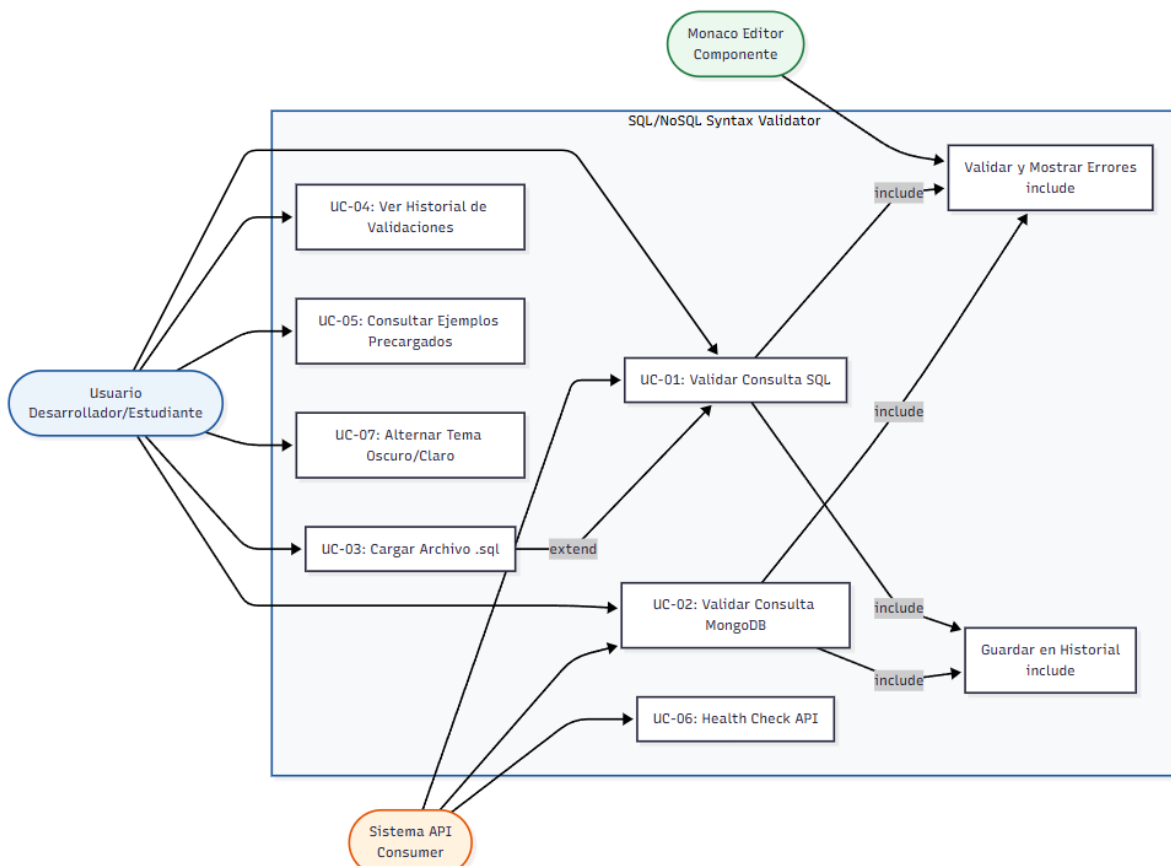
B. Modelo Conceptual

a. Diagrama de Paquetes





b. Diagrama de Casos de Uso



c. Escenarios Casos de Uso

UC-01: Validar Consulta SQL

ID	UC-01
Nombre	Validar Consulta SQL
Actor Principal	Usuario (Desarrollador/Estudiante)
Precondicion	El usuario tiene acceso a la interfaz web del validador
Flujo Principal	1. Usuario selecciona tipo 'SQL' en el selector de modo 2. Selecciona el dialecto: MySQL, PostgreSQL, SQLite o ANSI 3. Escribe o pega la consulta SQL en el editor Monaco 4. Presiona Ctrl+Enter o el boton 'Validar' 5. Sistema envia {type:'sql', query, dialect} al endpoint POST /api/validate 6. validation.service.js invoca node-sql-parser con el dialecto seleccionado 7. El parser retorna resultado {valid, errors, suggestions} 8. Sistema muestra resultado: indicador verde (valido) o rojo (error con posicion)
Flujo Alternativo	3a. Usuario presiona 'Subir Archivo' y selecciona un .sql: el editor carga el contenido automaticamente 8a. Error de red: sistema muestra mensaje 'Error de conexion con el servidor de validacion'
Postcondicion	Resultado de validacion mostrado en pantalla y guardado en historial local
Prioridad	Alta

UC-02: Validar Consulta MongoDB

ID	UC-02
Nombre	Validar Consulta MongoDB
Actor Principal	Usuario (Desarrollador/Estudiante)
Precondicion	El usuario tiene acceso a la interfaz web del validador
Flujo Principal	1. Usuario selecciona tipo 'NoSQL' en el selector de modo 2. El editor cambia a modo JSON con resaltado apropiado 3. Usuario escribe la consulta MongoDB en formato JSON 4. Presiona Ctrl+Enter o el boton 'Validar' 5. Sistema envia {type:'nosql', query} al endpoint POST /api/validate 6. validation.service.js valida: (a) estructura JSON, (b) comando MongoDB, (c) operadores 7. Retorna {valid, errors:[{message}], suggestions} 8. Sistema muestra resultado con errores semanticos especificos
Flujo Alternativo	6a. JSON invalido: sistema retorna error de parseo JSON sin continuar validacion semantica 6b. Comando no reconocido: sistema lista los comandos soportados en el mensaje de error

Postcondicion	Resultado de validacion mostrado y guardado en historial
Prioridad	Alta

UC-03: Cargar Archivo SQL

ID	UC-03
Nombre	Cargar Archivo SQL
Actor Principal	Usuario
Precondicion	El usuario tiene un archivo .sql en su sistema de archivos
Flujo Principal	1. Usuario presiona el boton 'Subir Archivo' en la barra lateral 2. Se abre el dialogo de seleccion de archivos del sistema operativo 3. Usuario selecciona un archivo con extension .sql 4. El contenido del archivo se carga en el editor Monaco 5. El modo del editor cambia automaticamente a SQL 6. El usuario puede proceder a validar la consulta cargada
Flujo Alternativo	3a. Usuario selecciona archivo con extension diferente a .sql: sistema muestra advertencia 'Solo se aceptan archivos .sql'
Postcondicion	Contenido del archivo visible en el editor, listo para validacion
Prioridad	Media

UC-04: Ver Historial de Validaciones

ID	UC-04
Nombre	Ver Historial de Validaciones
Actor Principal	Usuario
Precondicion	El usuario ha realizado al menos una validacion previa en el navegador
Flujo Principal	1. Usuario presiona el boton 'Historial' en la barra lateral 2. Sistema recupera el historial desde localStorage 3. Se muestra la lista de validaciones anteriores con fecha, tipo y resultado 4. Usuario hace clic en un item del historial 5. La consulta asociada se carga en el editor para su reutilizacion
Flujo Alternativo	2a. Historial vacio: sistema muestra mensaje 'No hay validaciones previas'
Postcondicion	Consulta historica cargada en el editor
Prioridad	Media

UC-05: Consultar Ejemplos Precargados

ID	UC-05
Nombre	Consultar Ejemplos Precargados
Actor Principal	Usuario
Precondicion	El backend esta disponible
Flujo Principal	1. Sistema carga automaticamente los ejemplos al iniciar la aplicacion via GET /api/examples 2. Usuario selecciona una consulta de ejemplo SQL o NoSQL del panel lateral 3. La consulta se carga en el editor con el modo correcto 4. Usuario puede validarla o modificarla
Flujo Alternativo	1a. Backend no disponible: sistema muestra ejemplos predefinidos en el frontend
Postcondicion	Consulta de ejemplo cargada en el editor
Prioridad	Baja

UC-06: Consultar Estado del Servidor (Health Check)

ID	UC-06
Nombre	Consultar Estado del Servidor
Actor Principal	Sistema / Desarrollador / Herramienta de Monitoreo
Precondicion	El backend esta en ejecucion
Flujo Principal	1. Cliente envia GET /api/health 2. Sistema retorna JSON con {status:'ok', message, timestamp, environment, uptime} 3. Cliente verifica que el estado sea 'ok' para confirmar disponibilidad
Flujo Alternativo	1a. Backend caido: cliente recibe error de conexion (timeout/connection refused)
Postcondicion	Cliente conoce el estado de salud del servicio
Prioridad	Baja

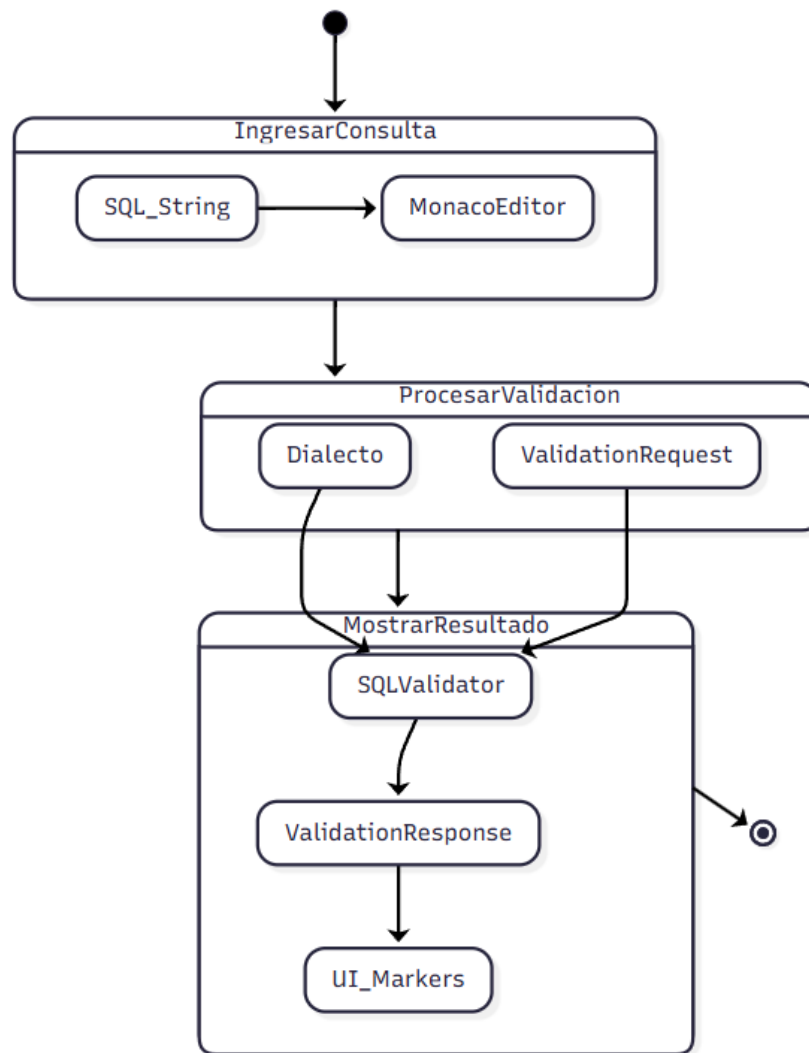
C. Modelo Lógico

a. Análisis de Objetos

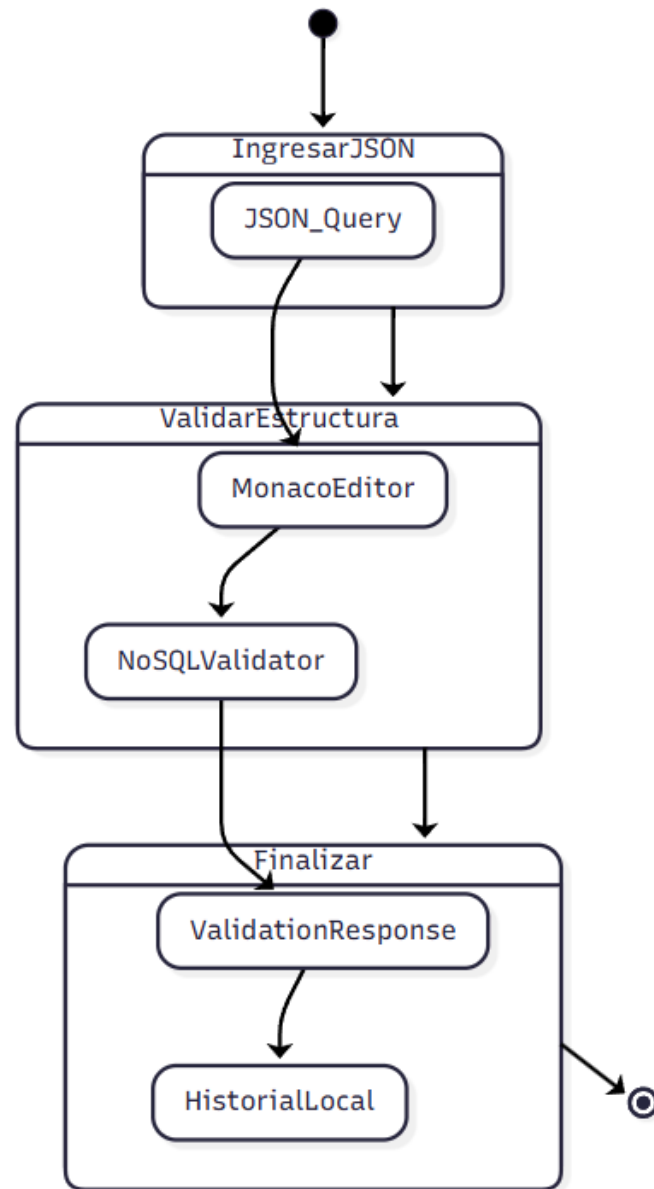
Clase/Modulo	Atributos / Propiedades	Metodos / Funciones
ValidationRequest	type: 'sql' 'nosql', query: string, dialect?: string	validate() — punto de entrada de validacion
ValidationResponse	valid: boolean, dialect?: string, errors: ErrorDetail[], suggestions: string[]	toJSON() — serializacion de respuesta
ErrorDetail	line?: number, column?: number, message: string	format() — formatea el mensaje de error
SQLValidator	parser: SqlParser, dialect: string	validate(query, dialect): ValidationResponse
NoSQLValidator	supportedCommands: string[], arrayOperators: string[]	validate(query): ValidationResponse, validateOperators(obj): ErrorDetail[]
ValidationService	sqlValidator: SQLValidator, nosqlValidator: NoSQLValidator	validate(request): ValidationResponse
ValidationController	validationService: ValidationService	handleValidate(req, res), handleExamples(req, res), handleHealth(req, res)
AppState (Frontend)	currentMode: 'sql' 'nosql', currentDialect: string, history: ValidationEntry[]	setMode(), setDialect(), addToHistory(), loadFromHistory()
MonacoWrapper (Frontend)	editor: monaco.editor.IStandaloneCodeEditor, markers: IMarkerData[]	initialize(), setLanguage(), setMarkers(errors), getValue()

b. Diagrama de Actividades con objetos

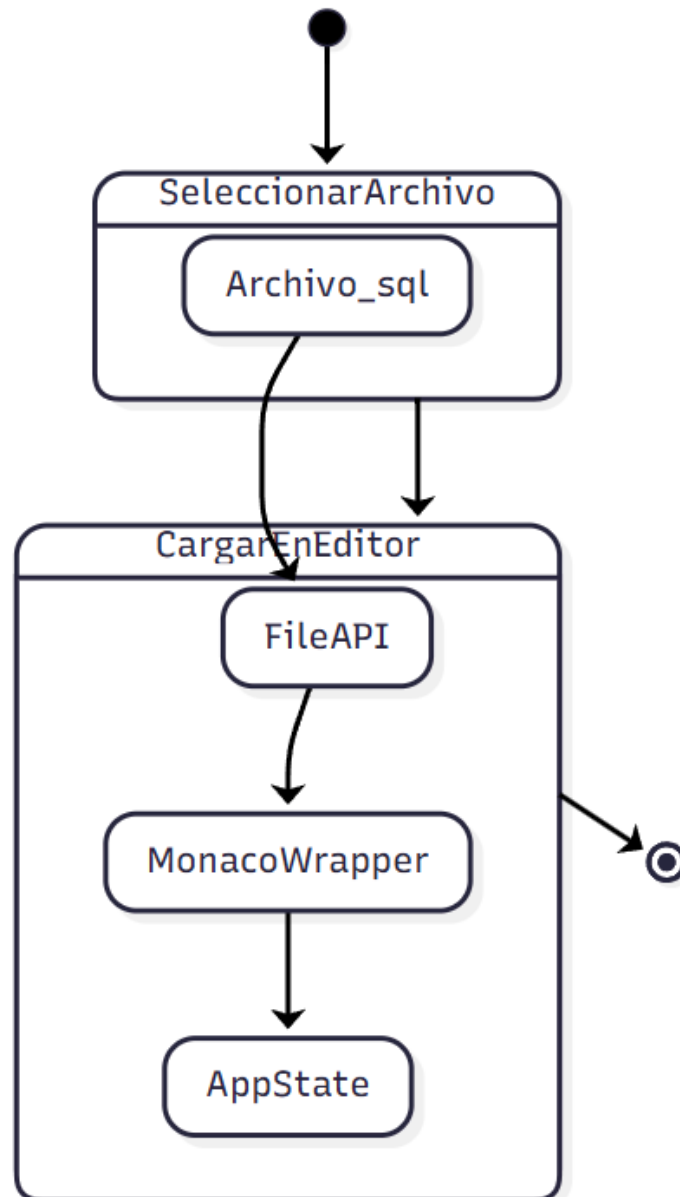
DA-UC01: Validar Consulta SQL

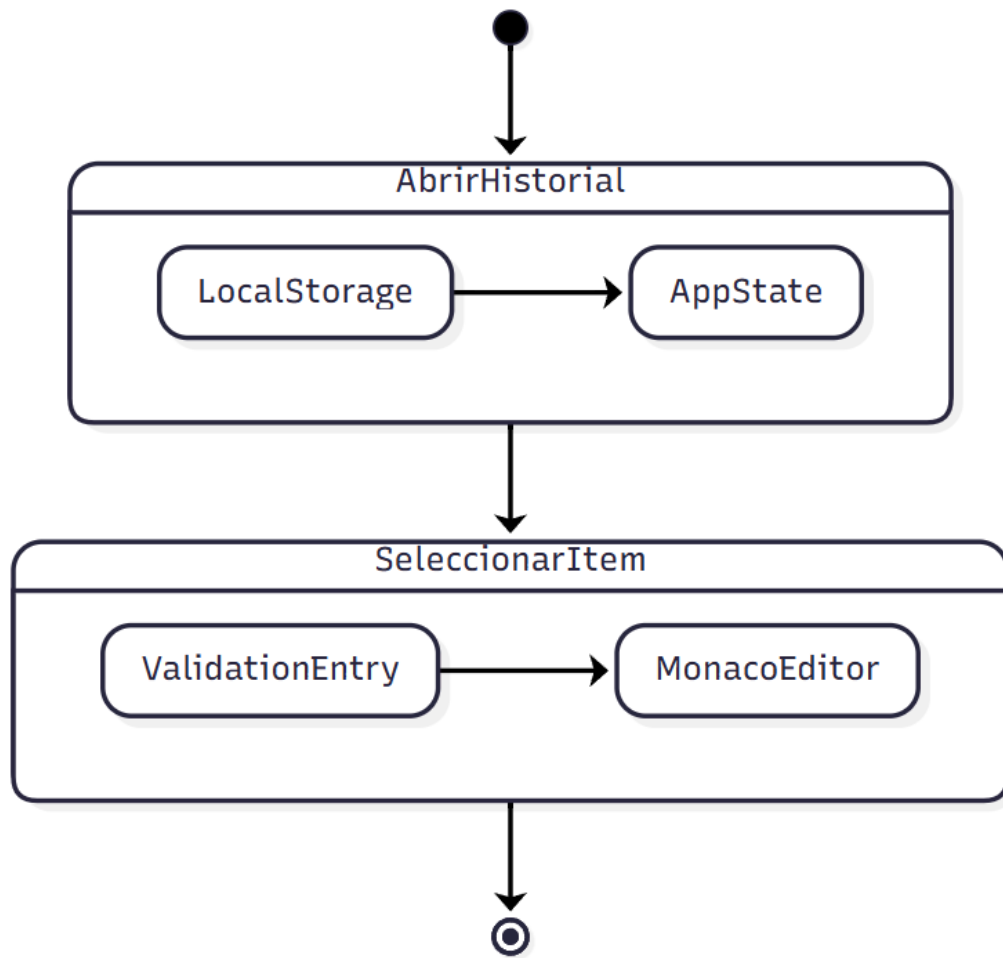


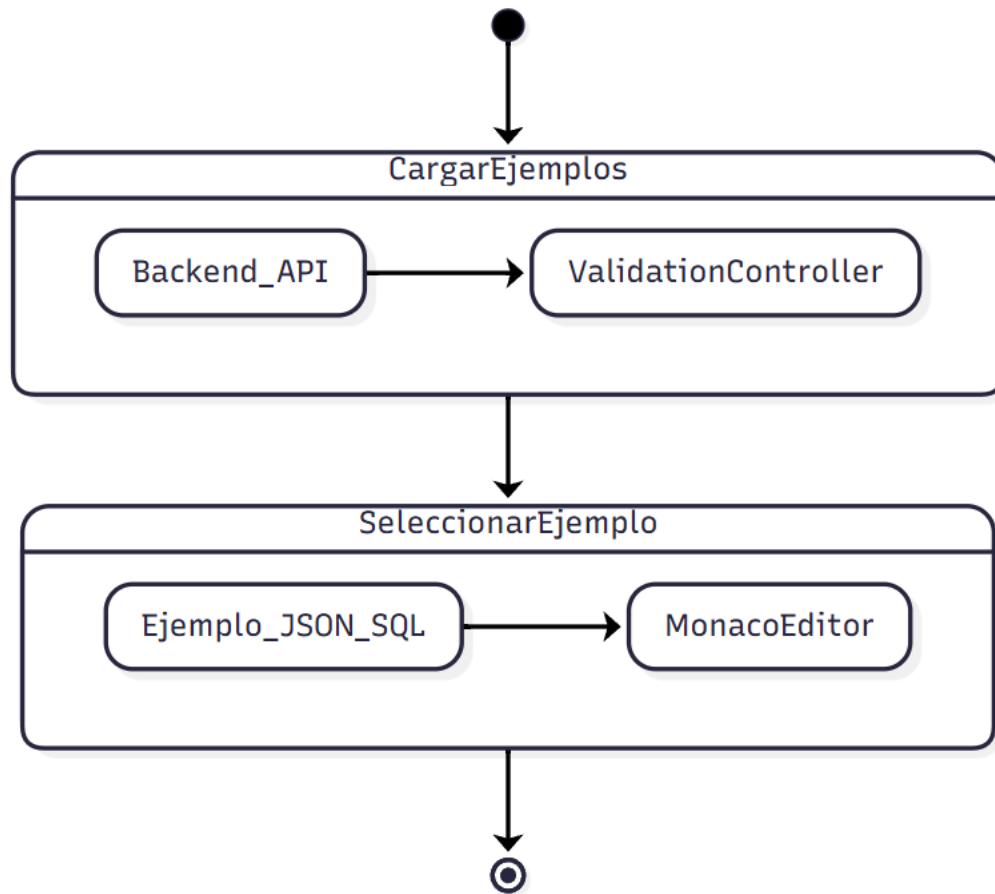
UC-02: Validar Consulta MongoDB

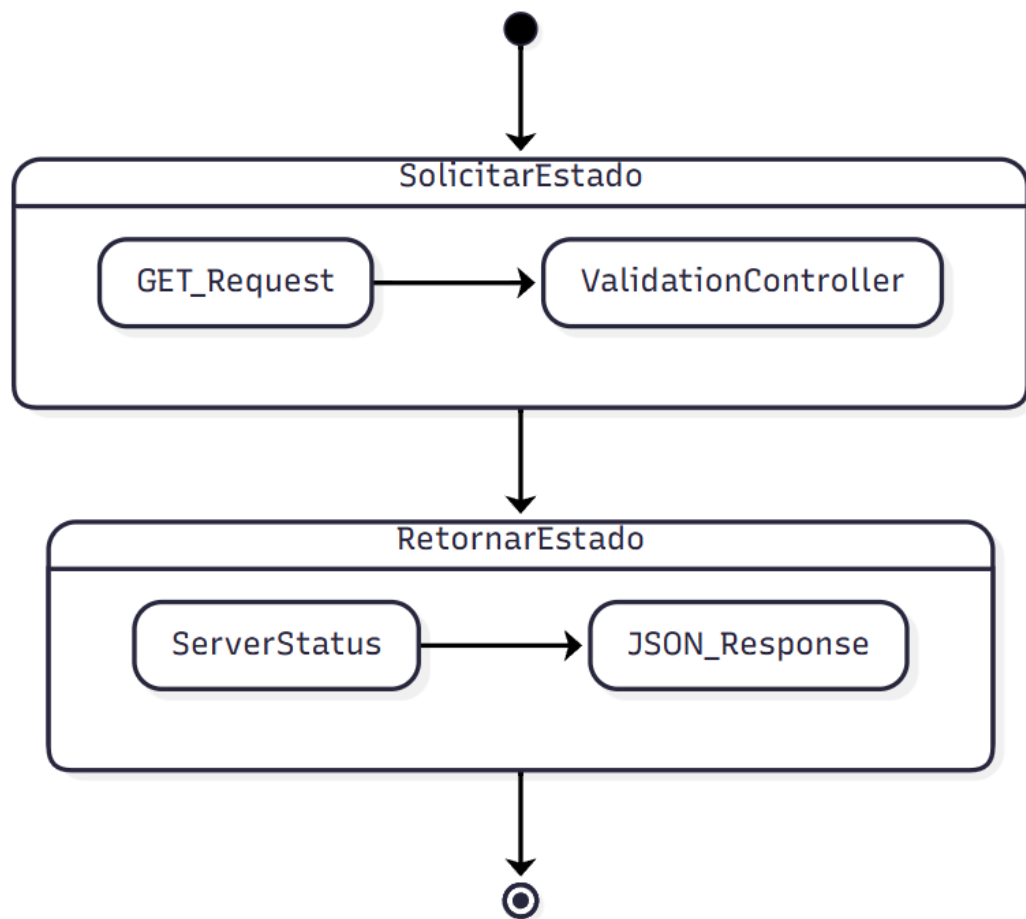


UC-03: Cargar Archivo SQL

**UC-04: Ver Historial de Validaciones**

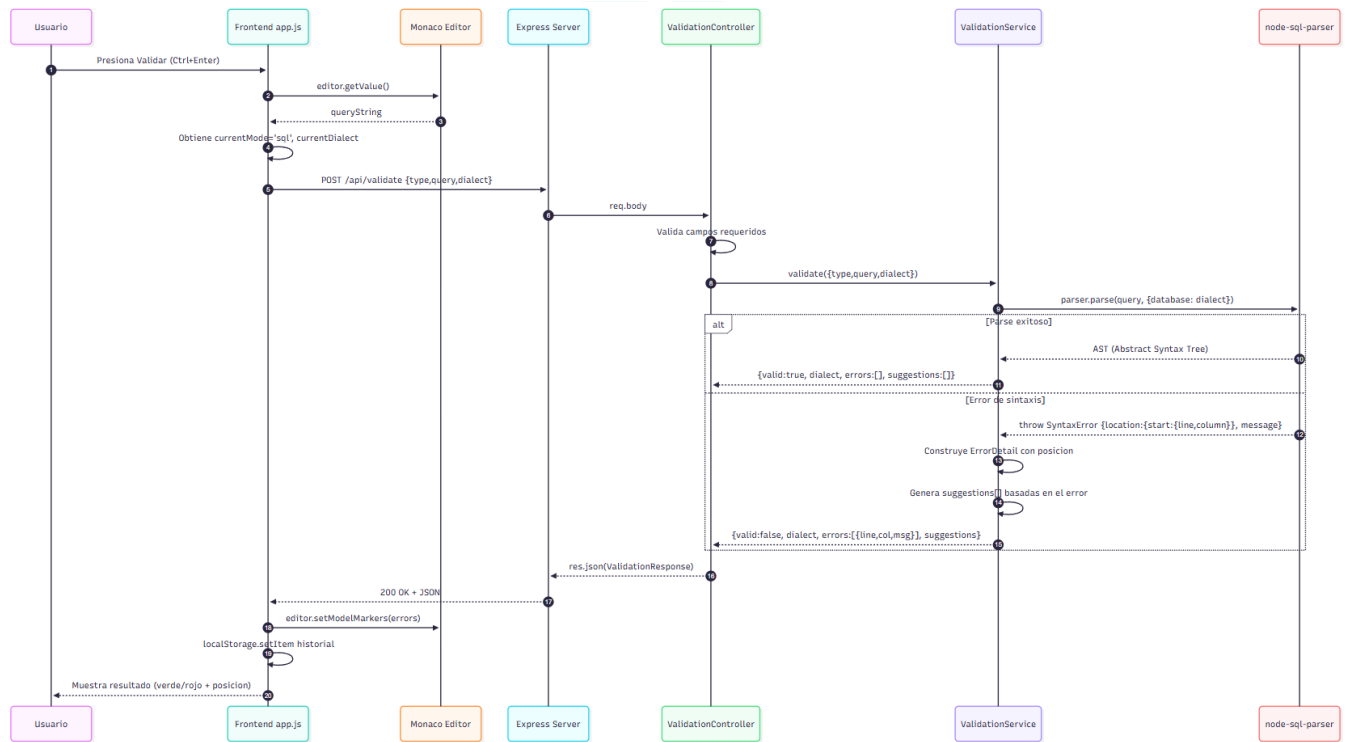
**UC-05: Consultar Ejemplos Precargados**

**UC-06: Health Check API**

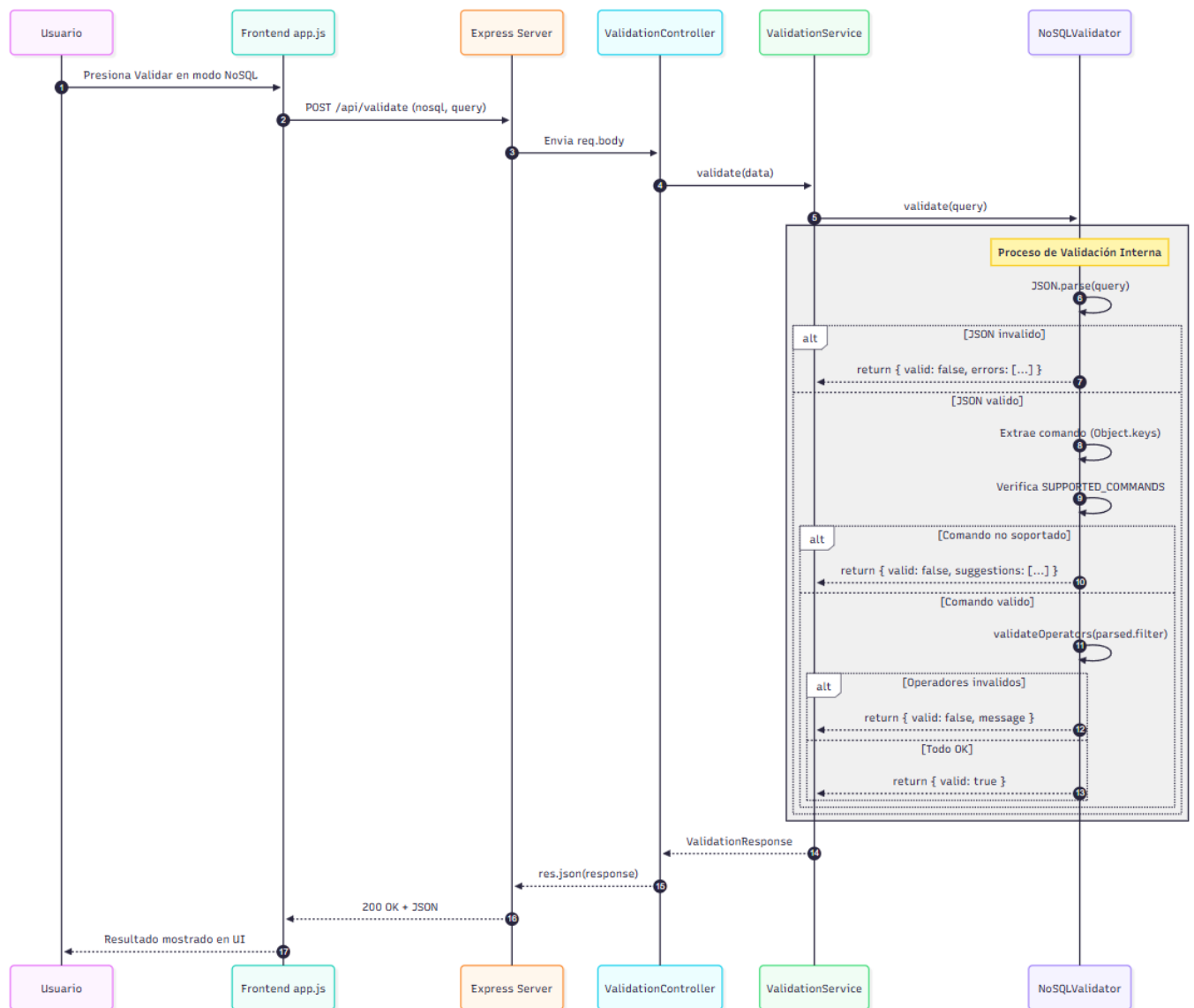


c. Diagrama de Secuencia

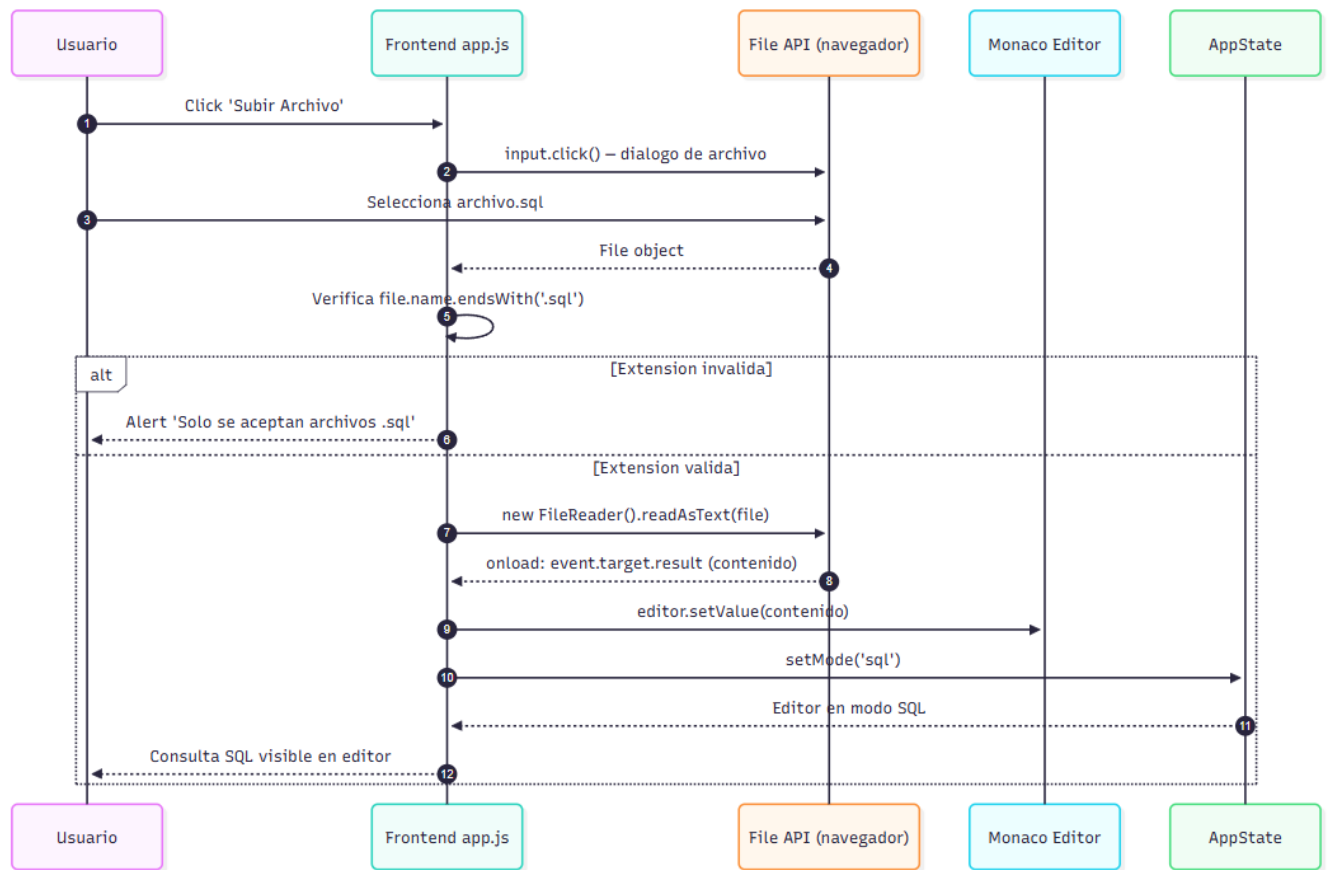
DS-UC01: Validar Consulta SQL



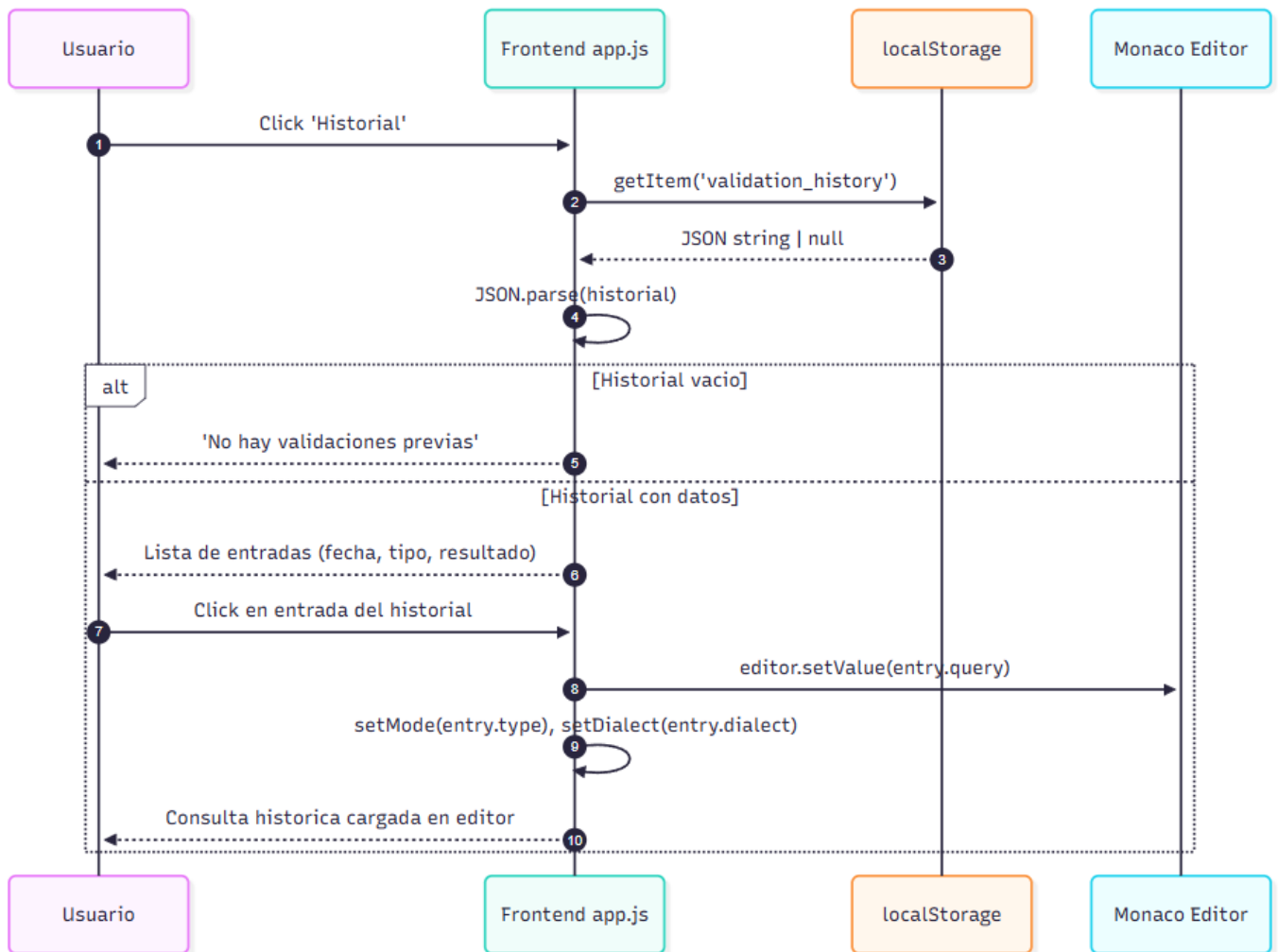
DS-UC02: Validar Consulta MongoDB

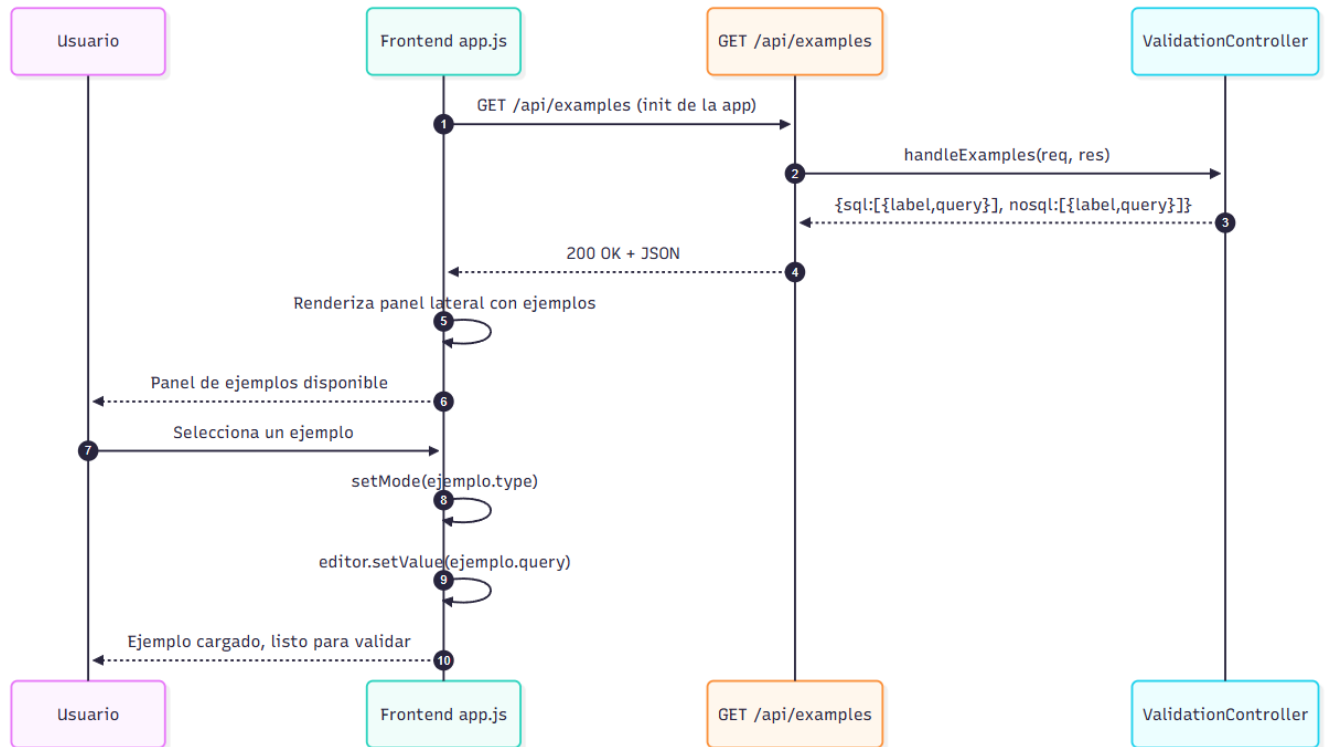


DS-UC03: Cargar Archivo SQL

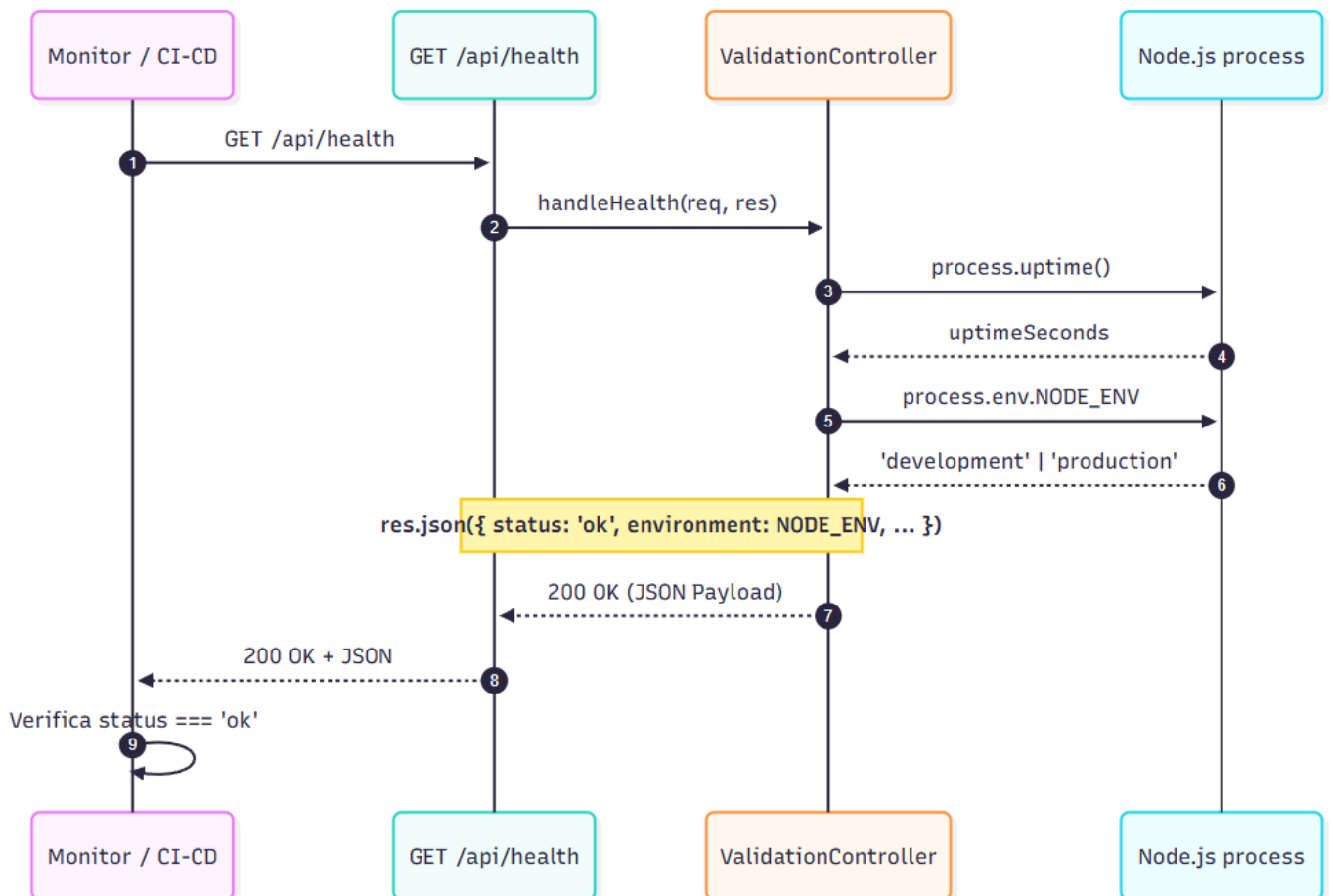


DS-UC04: Ver Historial

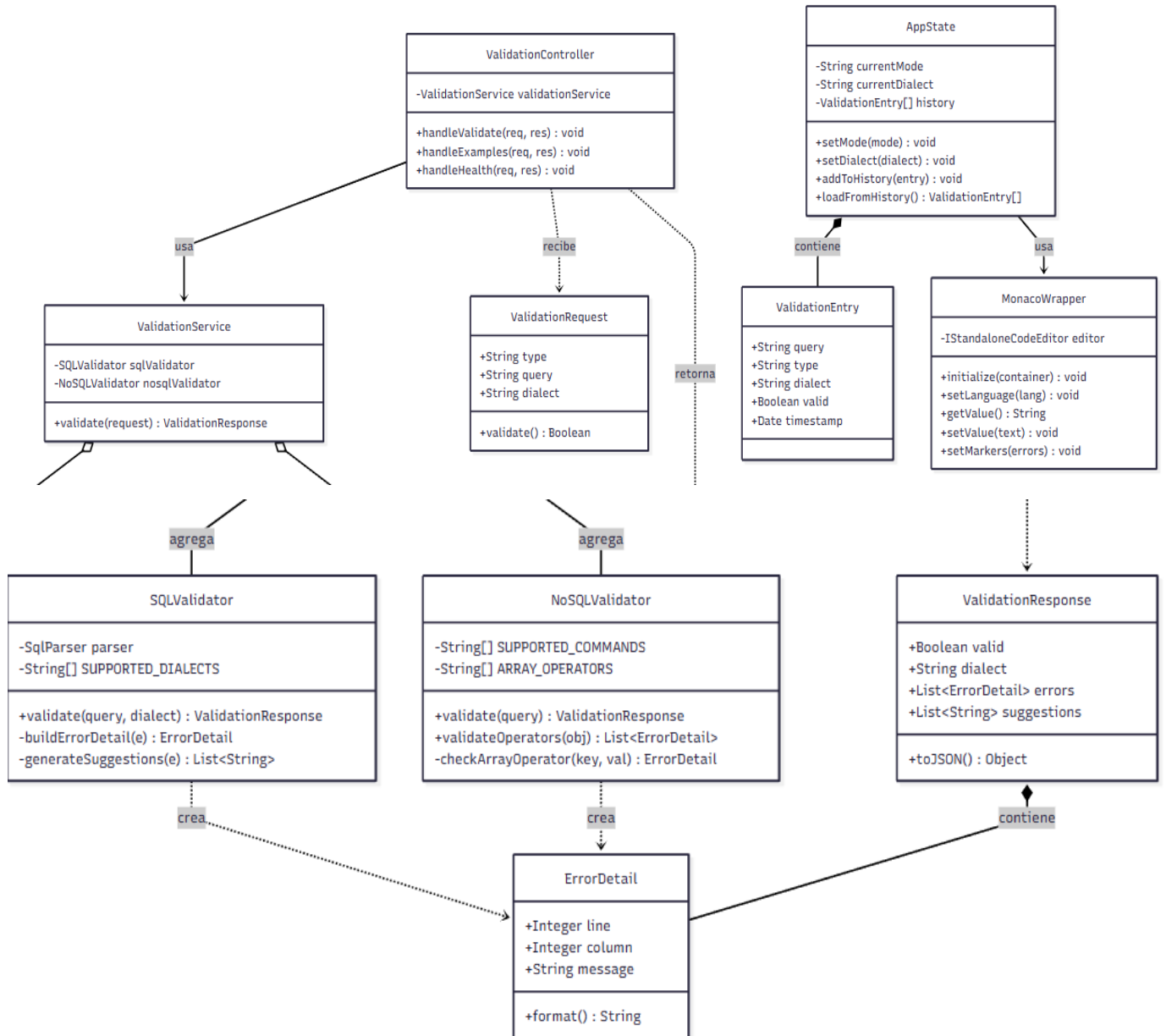
**DS-UC05: Consultar Ejemplos**



DS-UC06: Health Check



d. Diagrama de Clases



CONCLUSIONES

- El SQL/NoSQL Syntax Validator proporciona una solución efectiva para la validación de sintaxis de consultas de base de datos sin necesidad de un motor real, reduciendo significativamente el ciclo de detección de errores en el desarrollo de software.
- La arquitectura MVC con Node.js/Express garantiza una separación clara de responsabilidades que facilita el mantenimiento, las pruebas unitarias y la evolución independiente de cada capa del sistema.
- El uso de Monaco Editor como componente central de la interfaz proporciona una experiencia de usuario profesional comparable a entornos de desarrollo integrados, con marcado de errores en línea y resaltado de sintaxis.
- El soporte multi-dialecto (MySQL, PostgreSQL, SQLite, ANSI) y multi-paradigma (SQL relacional y MongoDB NoSQL) en una única herramienta diferencia al sistema de las soluciones existentes que suelen especializarse en un único motor.
- La exposición de una API REST permite la integración del validador en pipelines de CI/CD y herramientas de desarrollo externas, ampliando su utilidad más allá del uso interactivo.

RECOMENDACIONES

- Implementar pruebas unitarias para `validation.service.js` con Jest, cubriendo al menos 50 escenarios de SQL válido/inválido por dialecto y 30 escenarios MongoDB, para garantizar la regresión del motor de validación.
- Agregar soporte para SQL con múltiples sentencias (batch validation) para cubrir el caso de uso de archivos `.sql` con transacciones completas.
- Considerar la implementación de un worker de Node.js (`worker_threads`) para el procesamiento de consultas largas, evitando el bloqueo del event loop en archivos SQL de gran tamaño.
- Evaluar la integración de un linter de estilo SQL (similar a ESLint para JavaScript) que además de detectar errores sintácticos, sugiera mejores prácticas de escritura de consultas.
- Documentar la API REST con OpenAPI/Swagger para facilitar la integración por parte de consumidores externos del endpoint de validación.

BIBLIOGRAFÍA

- Google LLC. (2024). Android Developer Documentation — Security best practices. <https://developer.android.com/privacy-and-security/security-tips>
- C4 Model (2024). The C4 model for visualizing software architecture. Recuperado de: <https://c4model.com/>